Introduction to Digital Logic Design Lab
EECS 31L

Lab #02

Student Name: STEVEN KUZHIPALA

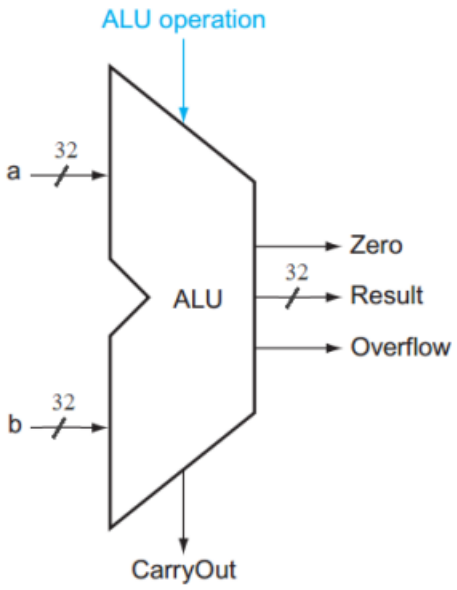
Student ID #: 51378962

Date: 2/6/26

ALU (Arithmetic Logic Unit)

1.Objective:

The objective of this lab is to create a 32-bit input ALU, that can handle both signed (for Additon, subtraction and set less than operations), and unsigned inputs. The block diagram for the ALU looks like such:



The ALU takes in 32-bit inputs, and puts out a 32 Bit output, along with 3 other 1 bits outputs:

- Zero: Goes high if the ALU output is all 0s

- Overflow: goes high If the result overflows the 32 bits range

- Carry Out: holds the carryout values that comes out of the Addition operation of the ALU, depending on the two inputs.

2.Procedure:

The logic for the AND, OR, NOR, EQUAL COMPARISSION operations are trivial, with just the use of the Verilog bitwise operator for all those operations.

The ADDITION, SUBTRACTION, SET LESS THAN, operations being signed, was performed with signed versions of the inputs, and for the ADDITION in specific to handle the carry_out

output, the operation was performed both in unsigned fashion(to get the carryout value), and in a signed manner (to calculate the appropriate ALU_OUT).

To handle the overflow conditions for the ADDITION, SUBTRACTION operations, two different formulas were constructed as follows:
(COMMENTS SHOW REASONING)

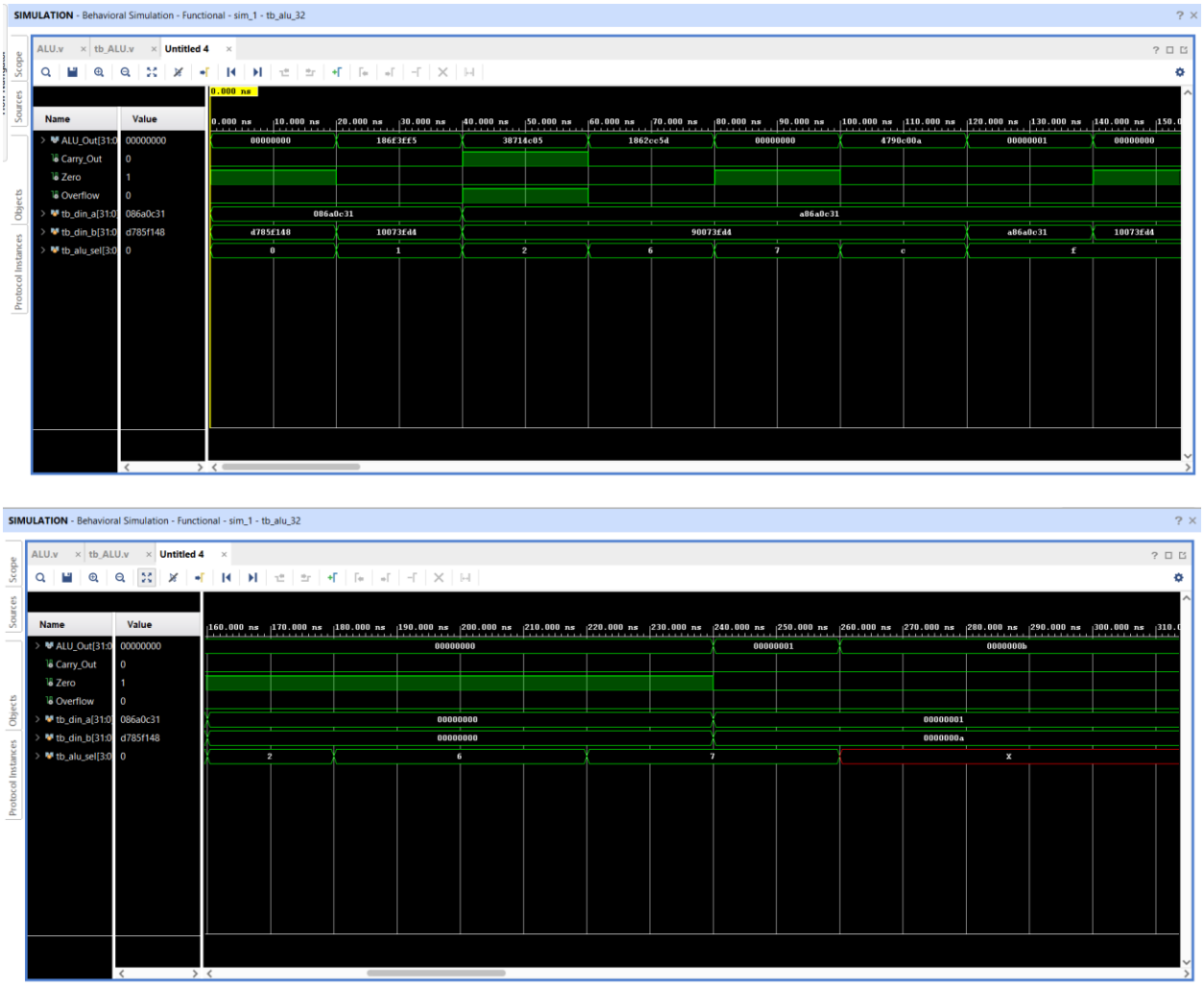
```
4'b0010: //ARTHEMETIC ADDITION - A SIGNED OPERATION
    begin
    unsign_A = A_in + B_in;
    Carry_Out= unsign_A[32]; //the last bit of the unassigned addition sum will be the carryOut
    ALU_Out = $signed(A_in) + $signed(B_in);
            //POSITIVE NUMBERS LEADING TO NEGATIVE OUTPUT     ;NEGATIVE NUMBERS LEADING TO POSITIVE OUTPUT
    Overflow = ( (!A_in[31] && !B_in[31] && ALU_Out[31]) ) || (A_in[31] && B_in[31] && !ALU_Out[31]);
    end  //end of Addition
```

```
4'b0110: //ARTHEMETIC SUBTRACTION - A SIGNED OPERATION
   begin
   ALU_Out = $signed(A_in) - $signed(B_in);
             //POSITIVE-NEGATIVE NUMBER=NEGATIVE NUMBER;  NEGATIVE-POSITIVE=POSITIVE OUTPUT
   Overflow= (!A_in[31] && B_in[31] && ALU_Out[31]) || (A_in[31] && !B_in[31] && !ALU_Out[31]);
   Carry_Out=1'b0;
   end  //end of subtraction
```

These formulas handled the two cases of overflow for each of the operations and allowed the ALU to handle signed inputs for these operations.

The carry out bit was only designed for the ADDITION operation, where a new 33 bit -reg value unsign_A was created, that held the sum of a unsigned addition of both the inputs, and its last bit was used as the Carry_Out output.

3.Simulation Results:



The output was as expected in the Lab Manual, and for the custom test cases written as well. When a unknown select line value was provided, the operation performed was addition, which has been specified for the default case.